

Modern SCADA Architecture and Development

Technical Report Based on the Architecture Discussion

Scope: current SCADA concepts, modern development trends, layered architecture, OPC UA, MQTT/Sparkplug, edge computing, Tag Engine, historian, alarms, commands, realtime communication, security, resilience, and a practical phased implementation.

Prepared: October 2026

Architecture direction: Web HMI + CodeIgniter application layer + MariaDB + dedicated long-running SCADA Core

Table of Contents

Table of Contents	2
1. Executive Summary	5
2. Current State of SCADA Development	5
2.1 From proprietary islands to interoperable platforms	5
2.2 Major development trends	5
3. Fundamental SCADA Architecture	5
3.1 Control remains in the PLC	6
4. Industrial Connectivity	6
4.1 Traditional protocols	6
4.2 OPC UA	6
4.3 Edge gateways	6
5. MQTT and Sparkplug	7
5.1 What MQTT solves	7
5.2 Why Sparkplug matters	7
5.3 Recommended first-version decision	7
6. The Tag Engine	7
6.1 Purpose	7
6.2 Tag definition versus runtime state	7
6.3 Naming	8
6.4 Data types and metadata	8
6.5 Quality	8
6.6 Current-value store	8
7. Historian Architecture	8
7.1 Purpose	8
7.2 Scale problem	8
7.3 Recording modes	8
7.4 MariaDB as an initial historian	9
7.5 Retention and aggregation	9
7.6 Asynchronous history writer	9
7.7 Historian abstraction	9
8. Alarm Engine	9
8.1 Events versus alarms	9
8.2 Alarm state machine	10

8.3 Alarm configuration	10
8.4 Deadband and delay	10
8.5 Shelving, suppression, and disabling	10
8.6 Alarm journal	10
8.7 Multi-operator synchronization	10
9. Command Engine	11
9.1 Command path	11
9.2 Server-side authorization	11
9.3 Validation	11
9.4 Command success versus process feedback	11
9.5 Command lifecycle	11
9.6 Audit	12
9.7 Safety boundary	12
10. Realtime Internal Architecture	12
10.1 Modular monolith first	12
10.2 Internal events	12
10.3 State versus durable events	12
10.4 WebSocket synchronization	13
10.5 No unnecessary internal broker	13
11. CodeIgniter and MariaDB Responsibilities	13
11.1 CodeIgniter application layer	13
11.2 MariaDB	13
11.3 What should not depend on ordinary PHP request/response execution	13
12. Reliability and Failure Behavior	14
12.1 Graceful degradation	14
12.2 Startup lifecycle	14
12.3 Health and heartbeat	14
13. Security Architecture	14
13.1 Zones and conduits	14
13.2 Security controls	15
14. Recommended Version 1 Architecture	15
15. Growth Path	16
15.1 Multi-site evolution	16
15.2 Potential future decomposition	16
15.3 Specialized historian	16

16. Implementation Principles	16
17. Suggested Next Engineering Decisions	16
18. Reference Standards and Technologies	17
19. Conclusion	17

1. Executive Summary

SCADA (Supervisory Control and Data Acquisition) is not merely a web dashboard connected to PLCs. A modern SCADA platform is a layered industrial system in which deterministic control remains in PLC/RTU or safety systems, while supervisory software acquires data, normalizes it into tags, evaluates alarms, stores history, distributes realtime state to HMIs, accepts authorized supervisory commands, and records an auditable operational history.

The recommended starting architecture is deliberately conservative: use a modular long-running SCADA Core for realtime functions; use a web application layer for authentication, configuration, reporting, administration, and APIs; use MariaDB for configuration, audit, alarms, and moderate historian workloads; and communicate with controllers through an acquisition layer such as OPC UA or Modbus.

For the first version, the report recommends avoiding unnecessary distributed-system complexity. Redis, Kafka, Kubernetes, and internal MQTT are not required merely because they are modern technologies. MQTT/Sparkplug becomes valuable when the platform grows to multiple sites, edge gateways, unreliable WAN connections, or multiple independent consumers of industrial data.

The core design principle is separation of responsibility: PLCs control the process; the SCADA Core supervises it; the historian stores time-series data; the alarm subsystem manages operator attention; the command subsystem validates supervisory actions; and the web layer provides secure user interaction.

2. Current State of SCADA Development

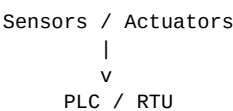
2.1 From proprietary islands to interoperable platforms

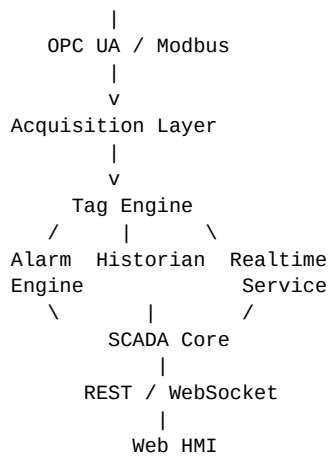
Traditional SCADA installations often coupled a specific PLC family, proprietary protocol, vendor HMI, and vendor historian. Current architectures increasingly favor open and interoperable interfaces, especially OPC UA, MQTT, REST-style application APIs, browser-based HMI technology, and standardized information models.

2.2 Major development trends

Trend	Meaning for SCADA development
OPC UA	Structured industrial data, subscriptions, security, information models, and interoperable device/application communication.
MQTT + Sparkplug	Publish/subscribe distribution with industrial conventions for topics, payloads, sessions, and device state.
Industrial Edge	Protocol conversion, local buffering, store-and-forward, filtering, and local analytics close to equipment.
Web HMI	Browser-based visualization using HTTPS, WebSocket, APIs, responsive UI, and centralized deployment.
Hybrid / Cloud	Local control remains local; cloud is used selectively for analytics, reporting, fleet management, AI, and enterprise integration.
Security by design	Network segmentation, identity, RBAC, certificates, TLS, audit, secure remote access, patching, and IEC 62443-oriented security.
Semantic models	Move from anonymous register numbers to meaningful assets, properties, relationships, and reusable models.
AI above the control loop	Anomaly detection, predictive maintenance, optimization, and operator assistance rather than direct safety control.

3. Fundamental SCADA Architecture





The PLC or RTU performs deterministic local control. The SCADA platform supervises, visualizes, historizes, alarms, and issues controlled supervisory commands. A failure of the web application or database should not disable the basic control logic of the physical process.

3.1 Control remains in the PLC

```

IF tank_level < 20%
  start pump

IF tank_level > 90%
  stop pump
  
```

Logic such as interlocks, sequencing, emergency behavior, and deterministic process control belongs in the PLC or appropriate safety system. The HMI can show the logic state and request actions, but should not be the only layer protecting equipment.

4. Industrial Connectivity

4.1 Traditional protocols

SCADA environments may encounter Modbus RTU/TCP, DNP3, EtherNet/IP, PROFINET, and vendor-specific protocols. These may expose low-level registers or coils that must be mapped to meaningful process variables.

4.2 OPC UA

OPC UA provides a stronger abstraction than raw register addressing. Instead of treating a pressure value as register 40002, the system can expose a structured node such as Plant01.Area01.Pump01.Pressure, including value, status/quality, timestamps, type information, and relationships.

```

Plant01
  Area01
    Pump01
      Pressure = 4.21 bar
      Flow     = 126 L/min
      Running  = true
      Speed    = 1450 rpm
  
```

OPC UA also supports subscriptions, allowing a client to receive monitored changes instead of continuously polling every value.

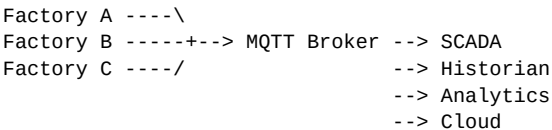
4.3 Edge gateways

Older controllers may not expose OPC UA directly. An industrial edge gateway can translate Modbus or proprietary protocols into OPC UA or MQTT, buffer data during outages, normalize values, and provide a controlled boundary between field networks and central services.

5. MQTT and Sparkplug

5.1 What MQTT solves

MQTT is especially useful when many producers and consumers need to exchange data without direct point-to-point integration. A broker decouples factories or edge gateways from SCADA, historians, analytics, AI, and enterprise applications.



5.2 Why Sparkplug matters

Plain MQTT defines messaging mechanics but not an industrial naming and state model. Sparkplug adds conventions for industrial topic namespaces, payloads, sessions, birth/death state, and related SCADA/IIoT behavior.

5.3 Recommended first-version decision

Do not require MQTT inside the first single-site SCADA Core. Use direct in-process events between internal modules. Introduce MQTT/Sparkplug when multi-site distribution, edge-to-central communication, WAN buffering, or multiple independent consumers justify it.

6. The Tag Engine

6.1 Purpose

The Tag Engine is the normalization center of the SCADA Core. It converts values from OPC UA, Modbus, MQTT, or other sources into a common internal representation that the rest of the platform can understand.

6.2 Tag definition versus runtime state

Tag Definition (configuration)	Runtime Tag State
ID and canonical name	Current value
Description and unit	Quality/status
Data type	Source timestamp
Device and source address	Server timestamp
Scan/update configuration	Connection/source state
Alarm and historian configuration	Latest normalized metadata

Definitions fit naturally in MariaDB because they change infrequently. Runtime state changes rapidly and should normally live in the long-running SCADA Core's current-state store.

6.3 Naming

```
Plant01.Area01.Pump01.Pressure
Plant01.Area01.Pump01.Flow
Plant01.Area01.Pump01.Running
Plant01.Area01.Tank01.Level
```

Hierarchical names improve navigation, permissions, filtering, alarm organization, historian queries, MQTT mapping, and future asset modeling.

6.4 Data types and metadata

- BOOL for discrete states such as Running.
- INT/UINT for counters and enumerations.
- FLOAT/DOUBLE for analog process values.
- STRING for limited textual state.
- DATETIME where the process explicitly exposes time values.
- Unit, source, quality, source timestamp, and server timestamp should be preserved where meaningful.

6.5 Quality

A displayed value is incomplete without quality. A pressure value of 4.21 bar can be trustworthy, uncertain, stale, or unavailable because communication has failed. The HMI must distinguish these cases rather than showing an old value as if it were live.

```
value           = 4.21
quality         = BAD_COMMUNICATION
source_timestamp = 10:20:00.123
server_timestamp = 10:25:15.900
```

6.6 Current-value store

The realtime service should maintain a fast current-state store so the HMI can request a snapshot without querying historical tables. The current store contains the latest normalized state for each tag.

7. Historian Architecture

7.1 Purpose

The historian stores process behavior over time for trends, reports, investigations, analytics, production analysis, and alarm context.

7.2 Scale problem

At one sample per second, 1,000 tags can generate 86.4 million samples per day. Therefore acquisition frequency, historian frequency, deadband, aggregation, retention, and query patterns must be designed separately.

7.3 Recording modes

Mode	Behavior
Cyclic	Store at a configured interval, such as every second.

Mode	Behavior
Change-based	Store when a value changes beyond a configured historian deadband.
Event-based	Store on operational events such as start, stop, alarm, batch start, or batch end.

7.4 MariaDB as an initial historian

MariaDB can be practical for small and medium workloads when the design uses compact tag IDs, suitable composite indexes, batch writes, time-oriented retention, aggregation, and carefully chosen sampling policies.

```
tag_history
-----
tag_id
timestamp
value
quality
```

A common query pattern is one tag over a time range, which makes an access path such as (tag_id, timestamp) natural.

7.5 Retention and aggregation

```
Raw / fine resolution  --> keep 30 days
    |
    v
1-minute aggregates    --> keep 1 year
    |
    v
1-hour aggregates      --> keep 10 years
```

Long-range HMI trends should not return millions of raw samples when the display has only a limited number of pixels. The historian API can select an appropriate resolution automatically.

7.6 Asynchronous history writer

```
Tag Engine
|
+--> Realtime HMI
+--> Alarm Engine
|
+--> Historian Queue --> Batch Writer --> MariaDB
```

Database latency should not block realtime acquisition. A durable bounded queue or equivalent persistence mechanism should absorb temporary database outages according to the required reliability level.

7.7 Historian abstraction

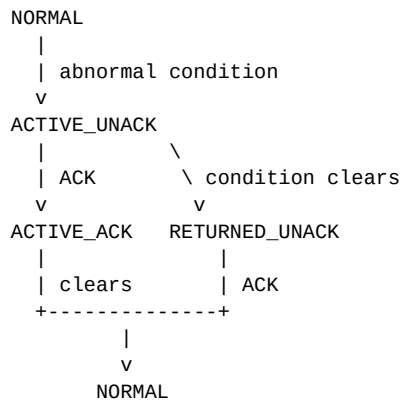
The Tag Engine should depend on a historian interface rather than MariaDB-specific logic throughout the codebase. This permits migration to a specialized time-series database or industrial historian later.

8. Alarm Engine

8.1 Events versus alarms

An event records something that happened. An alarm represents an abnormal condition requiring operator awareness or action. Not every event should become an alarm.

8.2 Alarm state machine



Acknowledgment means the operator has seen the alarm; it does not mean the abnormal process condition has been corrected.

8.3 Alarm configuration

- Threshold or condition.
- Priority.
- Deadband/hysteresis.
- Activation delay.
- Message and associated equipment.
- Enabled state and acknowledgment requirements.
- Suppression dependencies and shelving policy where applicable.

8.4 Deadband and delay

Deadband prevents alarm chatter around a threshold. Delay prevents insignificant short-duration excursions from creating nuisance alarms. Alarm deadband is distinct from historian deadband and process-control deadband.

8.5 Shelving, suppression, and disabling

Mechanism	Meaning
Disabled	Alarm configuration intentionally turned off.
Shelved	Authorized operator temporarily removes the alarm from normal presentation; should be time-limited, visible, and audited.
Suppressed	Process or equipment state makes the alarm currently inappropriate, such as low-flow alarm while a pump is intentionally stopped.

8.6 Alarm journal

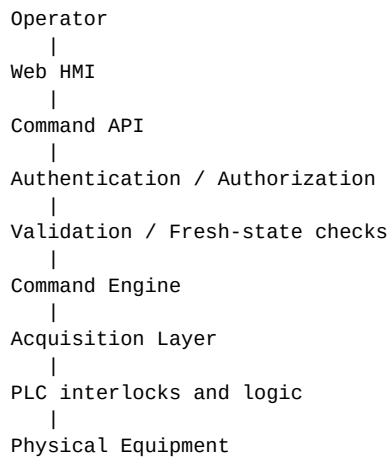
A durable event journal should record activation, acknowledgment, return-to-normal, shelving, unshelving, suppression changes, and other meaningful transitions. This is preferable to relying only on one mutable row representing the latest state.

8.7 Multi-operator synchronization

Alarm state is authoritative on the server. If one operator acknowledges an alarm, all connected HMIs should receive the resulting state change. The browser displays the alarm state; it does not own the alarm state machine.

9. Command Engine

9.1 Command path



The browser must never write arbitrary controller addresses directly. The server should expose configured commands such as Pump01.Start, Pump01.Stop, Valve01.Open, or Pump01.SpeedSetpoint.

9.2 Server-side authorization

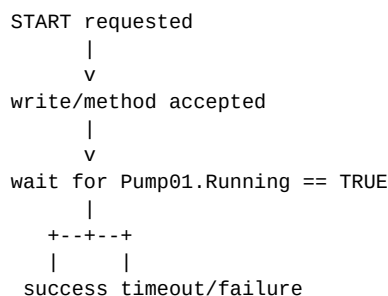
Hiding a button in the UI is not security. Every command request must be authenticated and authorized on the server. Permissions should be granular enough to distinguish viewing, alarm acknowledgment, operation, setpoint changes, configuration, and administration.

9.3 Validation

Setpoints require server-side type and range validation. Commands should also consider device connection state, tag quality, freshness, mode, and any supervisory prerequisites. The PLC still owns the real process interlocks.

9.4 Command success versus process feedback

A successful protocol write does not prove the equipment changed state. A pump start request may be accepted by communications while the PLC rejects it due to an interlock or the motor fails to start. The command subsystem should distinguish request status from authoritative process feedback.



9.5 Command lifecycle

State / Result	Meaning
REQUESTED	Operator or application submitted the request.
VALIDATED	Authorization and command validation succeeded.
SENT	Request was sent toward the controller.
ACCEPTED	Protocol/controller accepted the operation where such confirmation exists.
COMPLETED	Expected process feedback confirmed the intended outcome.
REJECTED_*	Authorization, validation, stale state, or device logic rejected the request.
TIMEOUT / COMMUNICATION_FAILURE	Unexpected result could not be confirmed.

9.6 Audit

Important commands should record user, time, workstation/session context, target equipment, requested value, old value when relevant, result, and feedback. Duplicate request protection or idempotency identifiers should prevent accidental repeated physical actions caused by retries or repeated clicks.

9.7 Safety boundary

Emergency stop and other safety functions should not depend solely on the browser, web application, database, or general SCADA server. Appropriate safety PLCs, safety circuits, and engineered protection layers must retain responsibility for safety-critical behavior.

10. Realtime Internal Architecture

10.1 Modular monolith first

Logical components do not need to be separate operating-system processes. For Version 1, a modular monolith is recommended for the realtime SCADA Core.

```
SCADA CORE SERVICE
-----
Acquisition Manager
Tag Engine
Alarm Engine
Command Engine
Historian Writer
Realtime / WebSocket Service
Internal Event Dispatcher
```

This keeps deployment, debugging, ordering, and failure analysis manageable while preserving module boundaries that can be split later if scale requires it.

10.2 Internal events

Modules can communicate through in-process events and queues. A normalized tag update can fan out to alarm evaluation, historian recording, and realtime publishing without forcing the acquisition path to wait for every downstream action.

10.3 State versus durable events

State	Durable Event
Latest value usually matters.	Occurrence and order matter.

State	Durable Event
Intermediate HMI values may be coalesced.	Alarm/command/audit transitions must not be casually dropped.
Fresh snapshot can rebuild client state.	Reconnect may require sequence-based catch-up or authoritative reload.

10.4 WebSocket synchronization

When an HMI opens a screen, it should subscribe to the required tags, receive an authoritative snapshot, and then receive incremental updates. After a disconnect, the client should visibly mark data as stale, reconnect, obtain a fresh snapshot, and resume updates.

10.5 No unnecessary internal broker

Redis and MQTT are not required merely for communication between modules in one SCADA Core process. They can be introduced later when independent processes, horizontal scaling, multi-site communication, or other concrete requirements justify them.

11. CodeIgniter and MariaDB Responsibilities

11.1 CodeIgniter application layer

- Authentication and session/token handling.
- Users, roles, and permissions.
- Plant, area, device, and tag configuration.
- Alarm configuration and alarm-history APIs.
- Reports and audit queries.
- HMI configuration and administration.
- REST APIs and integration with the realtime core.

11.2 MariaDB

- Users, roles, permissions, and configuration.
- Plants, areas, devices, tag definitions, command definitions.
- Alarm definitions and durable alarm/event history.
- Command/audit records.
- Reports and application metadata.
- Moderate historical time-series storage when workload permits.

11.3 What should not depend on ordinary PHP request/response execution

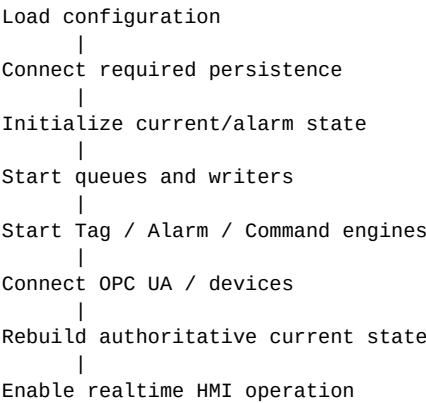
Continuous OPC UA subscriptions, long-running tag processing, alarm timers/state machines, high-rate historian queues, and persistent WebSocket distribution should live in a dedicated long-running service rather than relying on ordinary short-lived web requests.

12. Reliability and Failure Behavior

12.1 Graceful degradation

Failure	Desired behavior
MariaDB unavailable	PLC control continues; realtime core continues where possible; persistence buffers or enters a clearly degraded state.
Historian unavailable	Current monitoring continues; history is queued according to capacity/reliability policy.
WebSocket unavailable	PLC/control continue; browser realtime visualization is degraded/unavailable.
Browser closes	SCADA core and PLC continue normally.
Web application unavailable	PLC local control continues; realtime core should be independently survivable where practical.
OPC UA connection lost	PLC local control continues; affected tags become bad/stale and communication alarms are raised.
SCADA Core down	PLC local control continues; supervisory visibility/alarming/historization may be interrupted.

12.2 Startup lifecycle

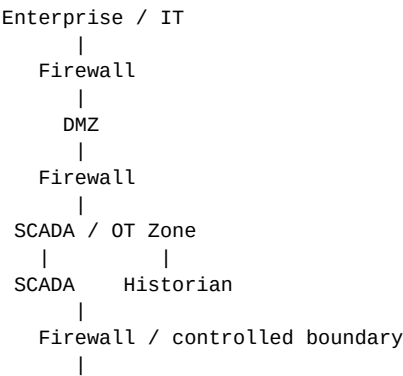


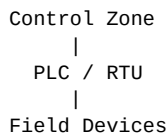
12.3 Health and heartbeat

The platform should expose health for the SCADA Core, historian writer, database connectivity, each industrial connection, queue usage, and storage pressure. Heartbeats help distinguish a quiet process from a disconnected edge or device.

13. Security Architecture

13.1 Zones and conduits





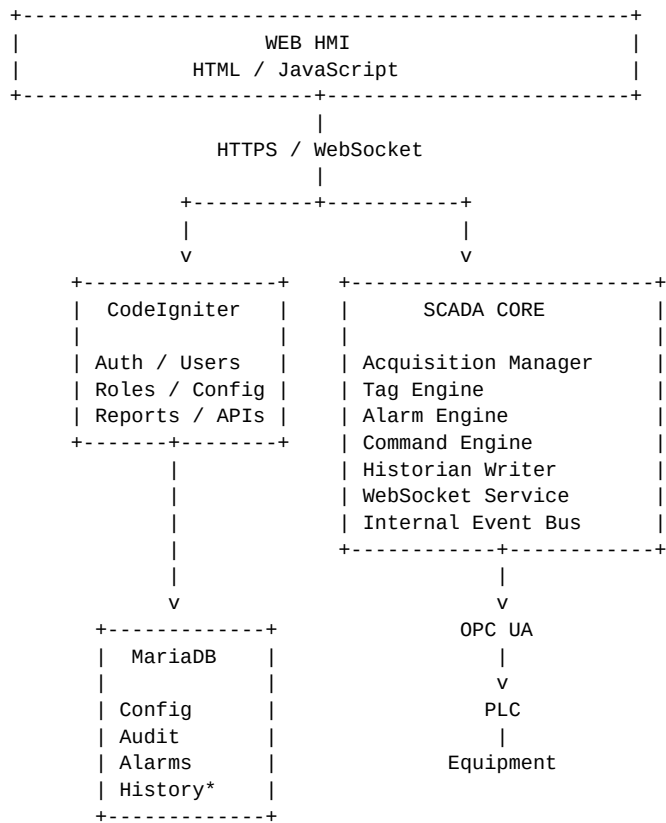
The exact network architecture depends on risk and site requirements, but direct Internet-to-PLC connectivity should not be the normal design.

13.2 Security controls

- Strong identity and authentication.
- Role-based authorization and least privilege.
- TLS and certificate management where supported.
- Network segmentation and controlled conduits.
- Secure remote access rather than uncontrolled inbound exposure.
- Audit logs for configuration, alarm acknowledgment, and commands.
- Asset inventory, patching, vulnerability management, and secure deployment.
- Signed software / secure boot where supported by industrial platforms.
- Explicit treatment of availability as a security requirement.

ISA/IEC 62443 is an important standards family for industrial automation and control-system cybersecurity and should inform the eventual security requirements and lifecycle.

14. Recommended Version 1 Architecture



* MariaDB history when workload permits.

This architecture keeps the first implementation understandable while preserving clear boundaries for future expansion.

15. Growth Path

15.1 Multi-site evolution

```
Plant A Edge ----\
Plant B Edge -----+--> MQTT/Sparkplug Broker --> Central SCADA
Plant C Edge ----/                               \--> Historian
                                                --> Analytics
                                                --> Enterprise/Cloud
```

MQTT/Sparkplug becomes more valuable when remote sites, unreliable WAN links, store-and-forward behavior, and multiple independent data consumers enter the requirements.

15.2 Potential future decomposition

If scale or organizational boundaries justify it, the modular SCADA Core can later be split into independent acquisition, alarm, historian, command, and realtime services. The Version 1 module interfaces should therefore be explicit even though they initially run in one process.

15.3 Specialized historian

As ingest rate, retention, compression, high availability, or analytics requirements grow, the historian interface can be backed by a purpose-built time-series or industrial historian without changing the conceptual Tag Engine or HMI model.

16. Implementation Principles

- Start with requirements and failure behavior, not with fashionable infrastructure.
- Keep deterministic control and safety functions in the appropriate PLC/safety layer.
- Treat value, quality, and timestamp as a unit of industrial data.
- Separate tag configuration from runtime state.
- Separate current state from historical time series.
- Treat alarms as server-side state machines, not red UI messages.
- Treat commands as audited workflows, not arbitrary writes.
- Use authoritative feedback to confirm physical outcomes.
- Do not let database latency block acquisition.
- Use snapshots plus incremental WebSocket updates for HMI synchronization.
- Preserve module boundaries even when using a modular monolith.
- Introduce MQTT/Sparkplug, Redis, or microservices only when requirements justify them.
- Design security, availability, audit, and recovery from the beginning.

17. Suggested Next Engineering Decisions

Decision	Questions to answer
SCADA Core language/runtime	C#, Java, Go, Rust, C++, Node.js, or another runtime; compare OPC UA libraries, concurrency, service deployment
OPC UA stack	Client library maturity, certificate handling, subscriptions, reconnect behavior, methods/writes, status code presentation
Historian limits	Expected tag count, update rate, retention, query patterns, acceptable loss window, aggregation strategy.
Web HMI stack	Component framework, SVG/process graphics, trend widgets, alarm banner, responsive layouts, accessibility.
Command model	Configured commands, feedback confirmation, timeout, duplicate protection, permissions, interlocks, audit.
Alarm philosophy	Priority model, alarm classes, deadbands, delays, suppression, shelving, acknowledgment, flood management.
Availability	Single server initially or redundant nodes; restart behavior; durable queue requirements; backup and restore.
Security	Zones/conduits, certificates, RBAC, remote access, patching, logging, secrets, deployment hardening.

18. Reference Standards and Technologies

The following public standards and technologies were central to the discussion and should be consulted during detailed engineering:

- OPC UA - OPC Foundation: industrial interoperability, information modeling, subscriptions, security, status/quality, timestamps, historical access, and methods.
- MQTT - lightweight publish/subscribe messaging used widely in IoT and increasingly in industrial edge architectures.
- Eclipse Sparkplug - industrial conventions on top of MQTT for SCADA/IIoT topic namespaces, payloads, and session/state behavior.
- ISA/IEC 62443 - cybersecurity lifecycle and requirements for industrial automation and control systems.
- ISA-18.2 - alarm management lifecycle and philosophy for process industries.
- MariaDB - relational storage suitable for configuration, audit, alarms, and potentially moderate historian workloads with appropriate design.

Useful official sites: opcfoundation.org, sparkplug.eclipse.org, isa.org, mariadb.com.

19. Conclusion

The recommended SCADA platform is not a single web application and database. It is a layered industrial architecture in which each subsystem has a clear responsibility. CodeIgniter and MariaDB remain useful choices for the management and application layer, while a dedicated long-running SCADA Core handles industrial connectivity, tags, alarms, commands, historian queues, and realtime HMI distribution.

For Version 1, simplicity is a strength: OPC UA/Modbus acquisition, a modular SCADA Core, server-side tag and alarm state, controlled command execution, MariaDB-backed configuration and moderate history, and a browser HMI over HTTPS/WebSocket. The design should keep interfaces clean so that future multi-site deployments can add industrial edge gateways, MQTT/Sparkplug, specialized historians, redundancy, analytics, and cloud integration without redesigning the fundamental model.

The most important architectural rule is that the supervisory software must fail safely and visibly: PLC control should continue independently, stale or unavailable data must be marked as such, alarms and commands must be auditable, and dependencies such as databases and browsers must not become hidden single points of physical control.